Estrutura de Dados não Lineares

UNIVERSIDADE DO VALE DO SAPUCAÍ – UNIVAS – POUSO ALEGRE/MG
PROFESSOR: RAFFAEL CARVALHO

Pilha: A Estrutura LIFO

O Conceito LIFO (Last In, First Out)

- **Definição:** O último elemento a entrar é obrigatoriamente o primeiro a sair.
- **Restrição:** Acesso restrito apenas a uma extremidade, chamada de Topo.
- **Analogias:**
  - Pilha de pratos.
  - Botão "Desfazer" (Ctrl+Z) — o último comando é o primeiro desfocado.
  - Pilha de execução de métodos (JVM Stack).

Operações Abstratas de uma Pilha

Para que uma estrutura seja considerada uma pilha, ela deve oferecer estas operações:

- **push(valor)**: Empilhar (adicionar no topo).
- **pop()**: Desempilhar (remover do topo e retornar o valor).
- **peek()** (ou top()): Espiar o topo (retornar o valor sem remover).
- **isEmpty()**: Verificar se está vazia.

Implementação Dinâmica: A Classe NoPilha

- Reaproveitamos a lógica da aula anterior: cada dado vive no Heap em um objeto Nó.
- A diferença é que a nossa lista só tem uma referência: o **topo**.

```
public class NoPilha {
    public int dado;
    public NoPilha proximo; // Aponta para o nó abaixo dele

    public NoPilha(int dado) {
        this.dado = dado;
        this.proximo = null;
    }
}
```

Operação push (Empilhar)

Lógica:

1. Criar novo nó.
2. O próximo do novo nó aponta para o atual topo.
3. O topo passa a ser o novo nó.

- **Complexidade: O(1) (Tempo constante).**

```
public void push(int valor) {
    NoPilha novoNo = new NoPilha(valor);
    novoNo.proximo = topo;
    topo = novoNo;
}
```

Operação pop (Desempilhar)

Lógica:

1. Verificar se está vazia.
2. Guardar o valor do topo.
3. O topo "pula" para o seu próximo (o nó de baixo).
4. Retornar o valor guardado.

Complexidade: O(1) (Tempo constante).

```
public int pop() {
    if (isEmpty()) throw new EmptyStackException();
    int valor = topo.dado;
    topo = topo.proximo; // O antigo topo flutua para o Garbage Collector
    return valor;
}
```

Resumo e Comparação

Estrutura	Inserção	Remoção	Acesso	Regra
Lista Ligada	O(1) no início, O(n) no fim	O(n) geral	O(n) sequencial	Livre
Pilha (Stack)	**O(1)** (Sempre no Topo)	**O(1)** (Sempre do Topo)	**O(1)** (Apenas Topo)	**LIFO**

Desafio Prático: Conversor de Decimal para Binário

Na computação, para converter um número inteiro decimal (base 10) para binário (base 2), nós dividimos o número sucessivamente por 2 e anotamos o resto da divisão (que será sempre 0 ou 1).O grande truque matemático é que o número binário final é formado lendo os restos de trás para frente (do último resto calculado até o primeiro).

Desafio Prático: Conversor de Decimal para Binário

A Missão:

1. Importe as classes NoPilha e PilhaDinamica que criamos em sala.
2. Crie uma nova classe chamada ConversorBinario com um método public static String converter(int numero).
3. Dentro desse método, use um laço de repetição (while) para dividir o número por 2.
4. **Empilhe** (push) cada resto da divisão na sua Pilha.
5. Quando o número chegar a zero, **Desempilhe** (pop) todos os itens da Pilha concatenando-os em uma String e retorne o resultado final.

Exemplo de Teste no seu App.java: Se você chamar converter(13), o sistema deve empilhar os restos: 1, depois 0, depois 1, depois 1. Ao desempilhar, a saída no console deve ser exatamente **1101**.

Bibliografia Básica

- CELES, Waldemar. Introdução a Estruturas de Dados - Com Técnicas de Programação em C. Grupo GEN, 2016. E-book. ISBN 9788595156654.
- PUGA, Sandra; RISSET, Gerson. Lógica de programação e estruturas de dados - com aplicações em Java. 2. ed. São Paulo: Pearson, 2006.
- GOODRICH, Michael T.; TAMASSIA, Roberto. Estruturas de dados e algoritmos em Java. Grupo A, 2013. E-book. ISBN 9788582600191.